

CROP YIELD PREDICTION

1. Overview

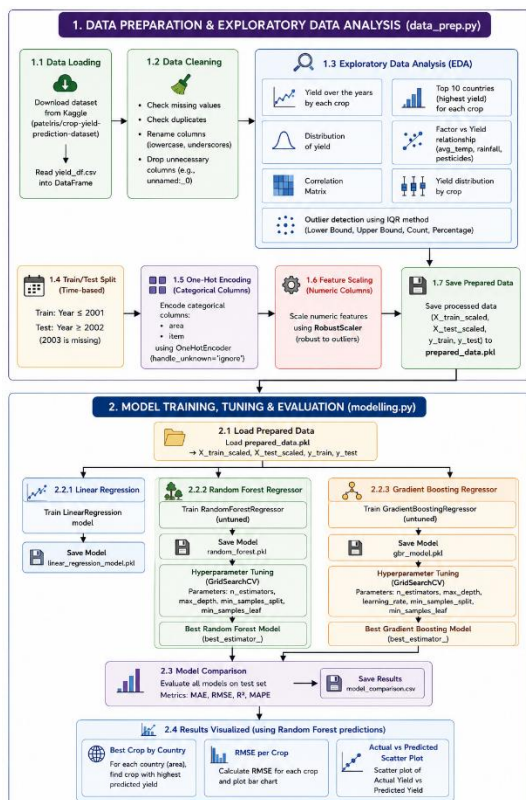
This project predicts crop yield (measured in hectograms per hectare) from a small set of inputs: the country, the crop, the year, and three environmental and agricultural factors, annual rainfall, pesticide use, and average temperature. It is a supervised regression problem built on a public Kaggle dataset covering 1990 to 2013 across many countries and ten crops.

The work is split across two scripts that run in order. `data_prep.py` loads the data, explores it, and converts it into a clean, model-ready form saved to disk and `modelling.py` loads that saved data, trains and compares several regression models, tunes the strongest ones, and produces the result tables and plots.

At a glance, the two scripts do the following:

Script	Responsibility
<code>data_prep.py</code>	Load, clean, explore (EDA), split, encode, scale, and save <code>prepared_data.pkl</code> .
<code>modelling.py</code>	Load <code>prepared_data.pkl</code> , train models, tune, evaluate, and write results and plots.

WorkFlow Diagram:



2. Dataset

Each row is one country / crop / year combination together with the yield recorded that year and the matching weather and input values. The data runs from 1990 to 2013 (2003 is missing) and covers ten crops.

Column	Description
area	Country the record belongs to (categorical).
item	Crop type - one of the 10 crops (categorical).
year	Year of the record, 1990–2013.
average_rain_fall_mm_per_year	Annual rainfall in millimetres.
pesticides_tonnes	Pesticide use in tonnes.
avg_temp	Average temperature.
hg/ha_yield	Target — yield in hectograms per hectare.

Two properties of the data shape everything that follows: yield is heavily right-skewed, and it varies far more between crops than it does with rainfall, pesticides, or temperature. In short, the crop itself is the strongest signal in the data.

3. What has been done

The full pipeline, end to end, is: download the data, clean and standardise it, explore it to understand the structure and relationships, split it by time into training and test sets, encode the categorical columns and scale the numeric ones, train a baseline plus two tree-based models, tune those two with cross-validated grid search, and finally evaluate and compare everything on the held-out years. Each stage is described in its own section below.

4. Exploratory Data Analysis (EDA)

Before any modelling, `data_prep.py` works through a full exploratory pass. The cleaning done first is light: column names are lowercased and spaces replaced with underscores, the stray index column is dropped, and the data is checked for missing values and duplicates (none of concern). The analyses that follow, and what each one showed, are summarised here.

Analysis	What it showed
Summary statistics	Yield ranges roughly 50 to 501,412 hg/ha with a mean near 77,053. A very wide spread.

Analysis	What it showed
Yield over time, by crop	Average yield per crop across 1990–2013, showing broad trends per crop.
Top producing countries per crop	Highest-yielding countries for each crop, filtered to countries with at least five records so small samples don't dominate.
Yield distribution	A histogram confirming yield is heavily right-skewed.
Factor vs yield	Rainfall, pesticides, and temperature plotted against yield (log scale). No clear individual relationship stands out.
Correlation heatmap	The numeric factors correlate only weakly with yield.
Yield by crop (boxplot)	Yield varies enormously from crop to crop — the dominant source of variation.
Outlier report (IQR)	Counts and percentages of outliers per numeric column using the 1.5×IQR rule.

The headline finding is that crop identity explains most of the variation in yield, while the three numeric factors are individually weak predictors. This is why the categorical columns matter so much in the model.

5. Train / Test Split

The split is done by time rather than at random. All rows up to and including 2001 are used for training, and 2002 onward is held out for testing.

Split	Years
Training set	1990 – 2001
Test set	2002 – 2013 (2003 absent)

This choice is deliberate. A random split would let the model see rows from the same years it is later tested on, effectively leaking future information and making the scores look better than they really are. A time-based split tests the model the way it would actually be used, predicting years it has never seen.

6. Feature Scaling and Encoding

Two transformations prepare the features for modelling. Both are fitted on the training set only and then applied to the test set, so no test information leaks into the fitting.

Categorical encoding

The two categorical columns, area (country) and item (crop), are one-hot encoded. Unknown categories are ignored at transform time, so an unseen country in the test set does not break prediction. The original columns are dropped and replaced with the encoded binary columns.

Numeric scaling

The four numeric columns — year, rainfall, pesticides, and average temperature are scaled with a RobustScaler. This scaler centres and scales using the median and interquartile range, which stands up to the skew and outliers in this data far better than a standard mean/standard-deviation scaler would.

The finished, scaled train and test features together with the two target vectors are saved to prepared_data.pkl, so the modelling stage can pick up exactly where preparation left off.

7. Model Training

modelling.py loads prepared_data.pkl and trains a baseline plus two tree-based ensembles. The baseline gives a reference point; anything more complex has to beat it to be worth keeping.

Model	Configuration
Linear Regression	Baseline, default settings.
Random Forest (untuned)	n_estimators = 200, max_depth = None, random_state = 42.
Gradient Boosting (untuned)	n_estimators = 200, max_depth = 5, learning_rate = 0.1, random_state = 42.

Each trained model is saved to its own .pkl file (linear_regression_model.pkl, random_forest.pkl, gbr_model.pkl) so it can be reused without retraining.

8. Tuning and Testing

Hyperparameter tuning

The random forest and gradient boosting models are tuned with GridSearchCV using 3-fold cross-validation, scored on negative root mean squared error. The search grids are:

Random Forest

Hyperparameter	Values tried
n_estimators	100, 200, 300
max_depth	None, 20, 40
min_samples_split	2, 5

Hyperparameter	Values tried
min_samples_leaf	1, 2

Gradient Boosting

Hyperparameter	Values tried
n_estimators	100, 200, 300
max_depth	3, 5, 7
learning_rate	0.05, 0.1, 0.2
min_samples_split	2, 5
min_samples_leaf	1, 2

Both searches are saved (rf_grid_result.pkl and gbr_grid_result.pkl). The first run performs the full search and saves the result; later runs load the saved result instead of retuning, so the expensive step only runs once. Deleting the cached files forces a fresh search.

Evaluation metrics

Every model — baseline, untuned, and tuned — is scored on the held-out test years using four metrics:

Metric	What it measures
MAE	Mean absolute error — average size of the error, in the same units as yield.
RMSE	Root mean squared error — like MAE but penalises large errors more heavily.
R ²	Proportion of the variance in yield the model explains
MAPE	Mean absolute percentage error — error relative to the actual value; sensitive to very small yields.

9. Results

It can be inferred that, Random forest untuned performed the best. Tuning both tree models did not make them any better.

Model Comparison

	Model	MAE	RMSE	R2	MAPE
	Random Forest (untuned)	13649.149	26197.194	0.919	21.609
	Random Forest (tuned)	13845.645	26790.403	0.915	21.723
	Gradient Boosting (untuned)	17263.368	29291.389	0.899	34.750
	Gradient Boosting (tuned)	21034.102	34714.666	0.858	43.674
	Linear(baseline)	31670.054	48107.931	0.726	81.889

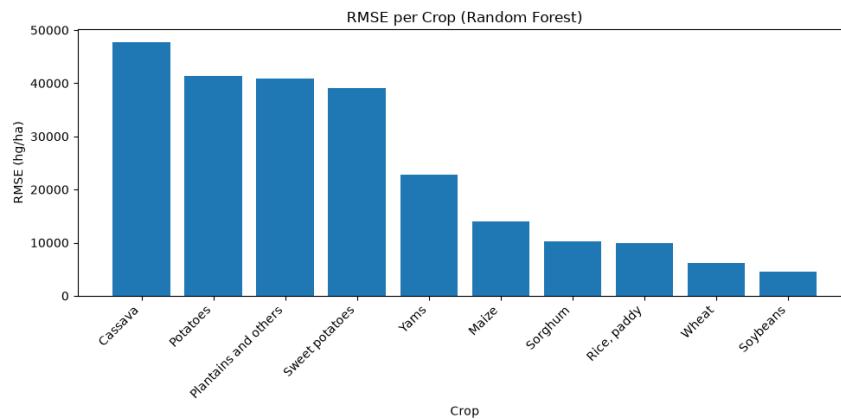
RMSE per crop

How well the chosen model predicts each crop.

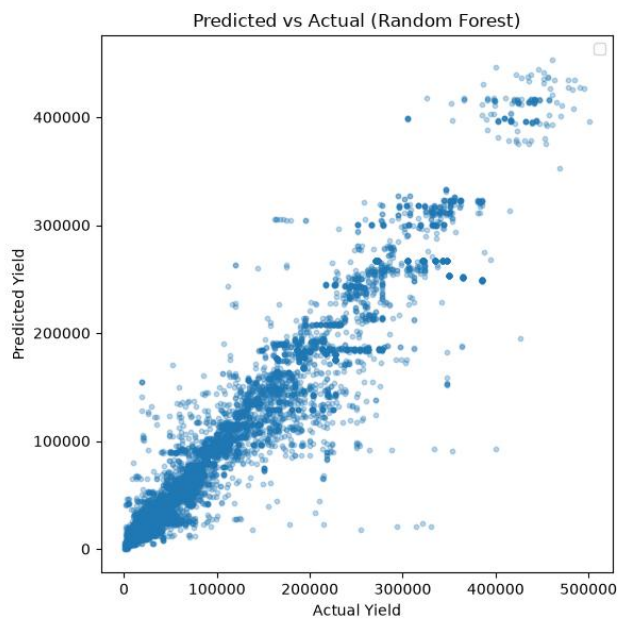
Item	RMSE
Cassava	47731.370103
Potatoes	41318.819930
Plantains and others	40932.947099
Sweet potatoes	39147.351943
Yams	22723.258076
Maize	14032.090555
Sorghum	10252.875055
Rice, paddy	9976.697441
Wheat	6181.651513
Soybeans	4521.496367

Plots produced

- RMSE per crop — bar chart of error by crop.



- Predicted vs actual yield — scatter plot with a reference line for perfect prediction.



10. Files Produced

File	Contents
prepared_data.pkl	Cleaned, encoded, scaled train/test features and targets.
linear_regression_model.pkl	Trained linear regression baseline.
random_forest.pkl	Trained (untuned) random forest.
gbr_model.pkl	Trained (untuned) gradient boosting model.
rf_grid_result.pkl / gbr_grid_result.pkl	Cached grid-search results.
model_comparison.csv	Metrics for every model, sorted by R^2 .

11. How to Run

- Install dependencies: `pip install -r requirements.txt`
- Set up Kaggle credentials for kagglehub (a `kaggle.json` token or the equivalent environment variables).
- Run `python data_prep.py` to build `prepared_data.pkl`.
- Run `python modelling.py` to train the models and generate the results.

The first modelling run is slow because of the grid searches. Every run after that reuses the cached results and is quick.

12. Notes

- The time-based split is intentional; switching to a random split would inflate the metrics by leaking future years.
- Because crop identity dominates yield, the per-crop RMSE is the more useful view of where the model struggles than a single overall number.
- MAPE can be inflated by rows with very small yields, so read it alongside the other metrics.
- The results section currently visualises predictions from the untuned random forest, since it showed the best performance.